TECHNICAL ARCHITECTURE •
DOC 3 / 3

User-Interface Architecture

The Flutter presentation layer of both iCare MC apps: the GetX pattern, navigation, module structure, theming, localisation and reusable widgets.

Subject	Patient app (icare_mc) & Doctor app (icare_doctors)
UI toolkit	Flutter (Material 3) compiled to AOT libapp.so
State mgmt	GetX — controllers, bindings, reactive Obx
Navigation	GetX named routes (GetPage / GetMaterialApp)
i18n	English, Amharic (■■■■), Afaan Oromoo
Method	Module/route/widget map recovered from AOT metadata

Contents

1	UI stack overview
2	The GetX architecture (MVC + reactive)
3	Navigation & routing
4	App shell & tab structure
5	Patient app module map
6	Doctor app module map
7	Reusable widget library
8	Theming & design system
9	Localisation & accessibility
10	UX patterns, feedback & native surfaces

1 UI stack overview

Both apps are single-`Activity` Flutter apps. The patient app hosts Flutter through a custom `FlutterActivity` variant, while the doctor app extends the stock `io.flutter.embedding.android.FlutterActivity`. All screens, layout, animation and interaction are drawn by the Flutter engine (`libflutter.so`) executing the app's compiled Dart (`libapp.so`). There are no Android XML layouts for the product UI — only Flutter widgets.

Key UI technologies (from bundled Dart packages)

Package	Role in the UI
`get (GetX)`	State management, DI, and named-route navigation
`Material 3`	Base widget set & theming (ColorScheme, Material)
`flutter_easyloading`	Global loading / toast overlays
`fluttertoast`	Lightweight toast messages (patient)
`pinput / otp_pin_field`	OTP entry UI
`country_code_picker`	Phone country selector (doctor)
`image_picker`	Camera/gallery for photos & documents
`flutter_inappwebview`	Embedded web content (patient)
`youtube_player_iframe`	In-app educational video playback (patient)
`chapasdk`	Native Chapa checkout screens (patient)
`cupertino_icons`	iOS-style iconography (doctor)

2 The GetX architecture (MVC + reactive)

Both apps follow the canonical **GetX pattern**, visible directly in the folder layout: every feature is a *module* split into `bindings/`, `controllers/` and `views/` (plus `widgets/`).

Layer	GetX type	Responsibility
View	GetView / StatelessWidget	Declarative UI; rebuilds reactively via Obx/GetX
Controller	GetxController	Screen state as reactive .obs fields; calls services/repos
Binding	Bindings	Lazily registers the controller + deps when a route opens
Service/Repo	Get.put/lazyPut singletons	API access (Supabase, HTTP), business logic

Reactivity uses observable fields and `Obx()` widgets — when a controller mutates an `.obs` value, only the wrapping `Obx` rebuilds. Dependency injection is handled by GetX bindings attached to each route, so a controller is created on navigation and disposed automatically when the route leaves the stack.

```
modules/dr_tab/
  bindings/doctor_booking_binding.dart     <- registers controller
  controllers/doctor_booking_controller.dart <- reactive state + logic
  views/doctor_booking_page.dart           <- Obx-driven UI
  widgets/dr_booking_card.dart             <- reusable sub-widgets
```

Consistent, testable structure

The strict bindings/controllers/views triad repeats across ~15 patient modules and ~14 doctor modules. This makes the codebase highly uniform: each screen is self-contained and its dependencies are declared in one place.

3 Navigation & routing

The root widget is a `GetMaterialApp`. Routes are declared centrally in `app/routes/app_pages.dart` as `GetPage` entries, each pairing a path, a view and a binding. Navigation is imperative and string-named (`Get.toNamed('/doctor-booking')`), wrapped by `core/utils/app_nav.dart`.

Recovered route names

Patient app routes	Doctor app routes
`/splash, /onboarding, /introduction`	`/splash, /sign-up, /login`
`/login, /sign-up, /forgot-password`	`/forgot-password, /main`
`/main (tab shell)`	`/dashboard (tab), /appointments`
`/profile, /edit-profile, /settings`	`/messages, /chat`
`/settings-baby / -language / -theme / -notifications`	`/availability`
`/health-tracker`	`/booking-services, /add-service`
`/doctor-detail, /doctor-service-detail`	`/service-reviews`
`/doctor-booking, /doctor-chat`	`/bank-details, /wallet-history`
`/my-appointments, /my-messages`	`/payment-history`
`/rating-screen`	`/personal-details, /verify-documents`

Deep-link handling is present via the `app_links` package (doctor app), allowing external URLs (e.g. payment returns) to route into the app.

4 App shell & tab structure

After authentication the user lands on a persistent **tab shell** (`main/views/main_shell_view.dart` driven by `main_shell_controller.dart`). The shell hosts a bottom navigation bar and swaps tab views while keeping their state alive.

Patient bottom tabs (from *_tab_view.dart)

- **Home** — daily greeting, tips, quick actions (`home_tab_view.dart`).
- **Baby / Child** — growth, milestones, vaccines (`baby_tab`, `child_tab`, `baby_growth`).
- **Pregnancy / Tools** — week-by-week content and calculators (`pregnancy_tab`, `pregnancy_dates`).
- **Doctors** — browse, book, chat, appointments (`dr_tab`).
- **Health tracker** — measurements & follow-ups (`health_tab`, `followup`).
- **Profile** — account, wallet, settings.

Doctor bottom tabs

- **Dashboard** — KPIs, monthly revenue chart, insights (`dashboard`, `monthly_revenue_chart.dart`).
- **Appointments** — upcoming/past bookings.
- **Messages** — patient conversations (`messages`, `chat`).
- **Payment history / Wallet** — earnings & payouts.
- **Profile** — personal details, verification, services, availability, bank details, settings.

> **Child-profile gate**
> The patient app guards child-specific tabs behind a `child_profile_gate.dart` / `baby_profile_setup.dart` flow: until a child (or pregnancy) is recorded, the relevant tabs prompt the user to set one up.

5 Patient app module map

Every module under `icare_mc/app/modules/` and its purpose:

Module	Screens / purpose
`splash / onboarding / introduction`	Launch, first-run walkthrough, about
`login / sign_up / forgot_password`	Phone-based auth & recovery
`main`	Bottom-tab shell container
`home`	Personalised dashboard & daily tips
`baby_tab / child_tab / baby_growth`	Child profile & growth charts
`development`	Milestones, learning path, add-measurement sheets
`health_tab`	Health tracker page & entry sheets
`followup`	Follow-up visits, vaccine alerts & banners
`pregnancy_tab / pregnancy_week`	Week-by-week pregnancy experience
`pregnancy_dates`	Due-date & pregnancy calculators/tools
`dr_tab`	Doctor browse, detail, service, booking, chat, ratings
`profile / settings`	Account, wallet, language/theme/notifications
`legal`	Terms & privacy viewer

6 Doctor app module map

Module	Screens / purpose
`splash / login / sign_up / forgot_password`	Launch & auth
`main`	Bottom-tab shell
`dashboard`	Revenue KPIs, insights, monthly chart
`appointments`	Manage patient appointments
`messages / chat`	Conversations with patients
`booking_services`	Create/manage services; add-service; reviews
`availability`	Set bookable time slots
`payment_history`	Completed transactions
`wallet_history`	Wallet ledger & payout tracking
`bank_details`	Chapa payout bank setup
`profile`	Personal details & document verification
`settings`	Theme, language, notifications
`legal`	Terms & privacy viewer

The doctor app adds finance-oriented UI (revenue chart, payout status, wallet history) and a credential-upload flow (`credential_image_picker.dart`, `verify_documents_view.dart`) that the patient app does not have.

7 Reusable widget library

Both apps ship a shared `app/widgets/` folder of composable UI building blocks, giving a consistent look across screens.

Shared / common widgets

- `primary_button`, `app_card`, `section_header`, `common_app_bar` — layout primitives.
- `app_text_field`, `phone_auth_field`, `otp_pin_field`, `date_picker_field` — form inputs (validated by `form_validators.dart`).
- `doctor_avatar`, `app_logo`, `daily_greeting_banner` — branded elements.
- `app_update_modal` — force/optional update prompt driven by the version-policy service.
- `chat/chat_widgets` — message bubbles / conversation tiles shared by chat screens.

Patient-specific widgets

- `child_avatar`, `child_birth_form_fields`, `record_birth_sheet`, `baby_profile_setup`.
- `milestone_media_view`, `pregnancy_week_hero_image`, `tool_grid_card`, `result_banner`.
- `gateway_payment_summary_card`, `guest_demo_banner`, `method_selector`.

Doctor-specific widgets

- `stat_card`, `monthly_revenue_chart` — dashboard analytics.
- `searchable_hospital_field`, `credential_image_picker`, `phone_link`, `profile_setting_widgets`.

8 Theming & design system

Theming is centralised in `themes/light_theme.dart` and `themes/dark_theme.dart`, coordinated by `core/theme/app_theme_controller.dart`. The controller toggles `ThemeMode` (light / dark / system) via GetX (`Get.changeThemeMode`) and persists the choice in local preferences.

- **Material 3** `ColorScheme`-based theming — the full Material 3 component theme set is present in the binary (app bar, cards, chips, dialogs, navigation bar, etc.).
- **Brand** assets: `app_branding.dart`, plus bundled `logo.png` / `logo_icon.png`.
- **Typography** via the default Material font with `MaterialIcons` icon font bundled in `FontManifest.json`.
- **User-selectable theme** exposed on the `settings-theme` screen.

> **System-driven surfaces**
> The app also sets Android system UI (status/navigation bar) via Flutter `SystemChrome`, and supports predictive-back gestures (Material predictive-back transitions are compiled in), aligning with modern Android 14+ behaviour (target SDK 36).

9 Localisation & accessibility

The apps are fully trilingual, a core requirement for the Ethiopian market. Locale is managed by `core/locale/app_locale_controller.dart` with strings in `core/locale/app_strings.dart` and a `services/localization_service.dart`.

Language	In-app label	Notes
English	English	Default / fallback
Amharic	■■■■	Full UI + content translations
Afaan Oromoo	Afaan Oromoo	Full UI + content translations

- UI strings switch instantly through GetX (`Get.updateLocale`); the choice is persisted and selectable on `settings-language`.
- **Content** (pregnancy weeks, tips, growth periods) is localised server-side via `*_translations` tables keyed by `language_code` and merged by `content_translation_service.dart`.
- Flutter `Semantics` and Material localisation delegates are present, providing baseline screen-reader support and locale-correct dates/number formatting (`date_format_utils.dart`).

10 UX patterns, feedback & native surfaces

10.1 Feedback & loading

- Global spinners/overlays via `flutter_easyloading`; toasts via `show_toast_dialog.dart` (`fluttertoast` on patient).
- Bottom sheets are used heavily for quick input (add-measurement, record-birth, health-tracker, learning-path sheets).
- Confirmation dialogs for destructive/financial actions (e.g. appointment cancellation with wallet-refund explanation).

10.2 Native / embedded surfaces

- **Chapa checkout** renders the SDK's native payment screens inside the patient flow.
- **YouTube playback** for educational content via `youtube_player_iframe` (`youtube_utils.dart`).
- **In-app webview** (`flutter_inappwebview`) for web content / legal pages.
- **Image capture** (`image_picker`) for child photos and doctor credential uploads.
- **Push notifications** surfaced through `flutter_local_notifications` on the 'ICare notifications' channel.

10.3 App-update UX

An `app_update_service.dart + app_version_policy_service.dart` compares the installed version against a server policy and against the store listing (Play Store URL, or `in_app_update` on the doctor app) to show an `app_update_modal` (optional or forced). A `MOCK_UPDATE_PROMPT` flag exists for QA.

Summary

The UI is a clean, uniform Flutter + GetX application: centralised named-route navigation, a per-feature bindings/controllers/views triad, a shared Material-3 design system with light/dark theming, trilingual localisation, and a rich reusable widget library. The patient app emphasises child-health and pregnancy journeys plus doctor booking, while the doctor app emphasises scheduling, messaging and finance. Native surfaces (Chapa, YouTube, webview, camera, notifications) are integrated through well-scoped Flutter plugins.

See companion documents: **API Architecture & Configuration** (1/3) and **Backend Architecture & Data Model** (2/3).